Лабораторна робота №10

«Локальне збереження даних у Flutter-застосунку»

Мета

Ознайомитися зі способами локального збереження даних у Flutter, навчитися зберігати та завантажувати дані користувача з екрана реєстрації, а також зберігати список обраних елементів (favorites ids) для подальшого відображення у застосунку.

Теоретичні відомості

1. Локальне збереження даних

Локальне збереження даних дає змогу зберігати інформацію без сервера, безпосередньо на пристрої користувача. Після повторного запуску застосунку ці дані можуть бути зчитані та знову використані в інтерфейсі.

У цій лабораторній роботі локально повинні зберігатися:

- дані користувача, введені на екрані реєстрації;
- список ідентифікаторів обраних елементів (favorites ids).

2. Які саме дані потрібно зберігати

У межах цього завдання потрібно зберігати:

Дані користувача з екрана реєстрації:

- ім’я користувача;
- email;
- пароль або інше поле, якщо воно передбачене структурою застосунку.

Дані обраних елементів:

- список id елементів, які користувач позначив як улюблені.

Такий підхід дозволяє:

- після повторного запуску не проходити реєстрацію повторно;
- відновлювати профіль користувача;
- автоматично показувати раніше збережені улюблені елементи.

3. SharedPreferences для простих даних

SharedPreferences зручно використовувати для:

- імені;
- email;
- пароля;
- списку favorites ids, якщо він зберігається як List<String>.

Приклад збереження даних користувача

```
import 'package:shared_preferences/shared_preferences.dart';


Future<void> saveUserData({
 required String name,
 required String email,
 required String password,
}) async {
 final prefs = await SharedPreferences.getInstance();
 await prefs.setString('name', name);
 await prefs.setString('email', email);
 await prefs.setString('password', password);
}
```

Приклад отримання даних користувача

```
Future<Map<String, String>> loadUserData() async {
```

```
 final prefs = await SharedPreferences.getInstance();
 return {
   'name': prefs.getString('name') ?? '',
   'email': prefs.getString('email') ?? '',
   'password': prefs.getString('password') ?? '',
 };
}
```

4. Збереження favorites ids

Улюблені елементи зручно зберігати як список ідентифікаторів.

Приклад збереження favorites ids

```
Future<void> saveFavoriteIds(List<String> favoriteIds) async {
 final prefs = await SharedPreferences.getInstance();
 await prefs.setStringList('favorite_ids', favoriteIds);
}
```

Приклад отримання favorites ids

```
Future<List<String>> loadFavoriteIds() async {
 final prefs = await SharedPreferences.getInstance();
 return prefs.getStringList('favorite_ids') ?? [];
}
```

Якщо id у моделі мають тип int, їх можна перед збереженням перетворити у рядки.

```
final ids = [1, 2, 3];
final stringIds = ids.map((e) => e.toString()).toList();
```

5. Використання збережених даних у застосунку

Після запуску застосунку потрібно:

- завантажити дані користувача;
- завантажити список favorites ids;
- відобразити дані на екрані профілю;

- показати у Favorites лише ті елементи, id яких є у списку збережених.

  Приклад завантаження у initState

```
@override
void initState() {
 super.initState();
 loadUserData();
 loadFavoriteIds();
}
```

Завдання на лабораторну роботу

1. Додати у проєкт пакет для локального збереження даних (SharedPreferences або інший аналогічний інструмент).
2. Реалізувати збереження даних користувача з екрана реєстрації.
3. Зберігати такі поля:
   a. ім’я користувача;
   b. email;
   c. пароль або інше поле, передбачене формою реєстрації.
4. Реалізувати завантаження даних користувача після повторного запуску застосунку.
5. Відобразити збережені дані на екрані ProfileScreen.
6. Реалізувати додавання елементів у список обраного.
7. Зберігати локально лише список favorites ids.
8. Після запуску застосунку автоматично відновлювати список обраних елементів за збереженими id.
9. На екрані FavoritesScreen відображати лише ті елементи, id яких містяться у списку збережених.

10. Перевірити, що після перезапуску застосунку:
    - дані профілю не втрачаються;
    - обрані елементи залишаються в списку Favorites.

Контрольні запитання

1. Що таке локальне збереження даних у Flutter?
2. Які дані в цій лабораторній роботі потрібно зберігати локально?
3. Для чого використовується SharedPreferences?
4. Як зберегти ім’я та email користувача локально?
5. Як зберегти список favorites ids?
6. Чому для обраних елементів доцільно зберігати саме id, а не весь об’єкт?
7. Як отримати збережені дані після запуску застосунку?
8. Для чого використовується initState() при роботі з локальними даними?
9. Як відобразити збережені дані користувача на ProfileScreen?
10. Як відновити список Favorites після перезапуску застосунку?
11. Які переваги локального збереження даних для мобільного застосунку?
12. Які помилки можуть виникнути, якщо не перевіряти наявність збережених даних?